

Hoja de Trabajo 2

Diego Fabian Morales Davila

Parte 1

```
fabim@JostMachine:/mnt/c/Users/fabim/OneDrive/Desktop/UVG/Semestre8/CompuParalela/Hoja2$ mpicc parte1.c -o parte1
fabim@JostMachine:/mnt/c/Users/fabim/OneDrive/Desktop/UVG/Semestre8/CompuParalela/Hoja2$ mpiexec -n 4 ./parte1
Rank 0 comunicador
Rank 1 con size 4
Rank 2 con size 4
Rank 3 con size 4
fabim@JostMachine:/mnt/c/Users/fabim/OneDrive/Desktop/UVG/Semestre8/CompuParalela/Hoja2$ mpiexec -n 1 ./parte1
Rank 0 comunicador
```

1. El rank es el identificador de cada proceso dentro del comunicador y va de 0 hasta el total de procesos menos uno, mientras que el size es la cantidad total de procesos que se lanzaron con mpiexec. Mi computadora tiene 16 núcleos, así que el programa puede correr con hasta 16 procesos; en la captura se usaron 4. El rank 0 funciona como comunicador y recibe el rank de los demás procesos para imprimirlos en orden.
2. Con 1 núcleo solo existe el rank 0 con size 1, por lo que únicamente se imprime "Rank 0 comunicador" ya que no hay otros procesos de los cuales recibir mensajes.

Parte 2

```
fabim@JostMachine:/mnt/c/Users/fabim/OneDrive/Desktop/UVG/Semestre8/CompuParalela/Hoja2$ mpicc parte2.c -o parte2
fabim@JostMachine:/mnt/c/Users/fabim/OneDrive/Desktop/UVG/Semestre8/CompuParalela/Hoja2$ mpiexec -n 4 ./parte2
Rank 0: Etapa 1, antes de la barrera
Rank 2: Etapa 1, antes de la barrera
Rank 3: Etapa 1, antes de la barrera
Rank 1: Etapa 1, antes de la barrera
Rank 3: Etapa 2, después de la barrera
Rank 0: Etapa 2, después de la barrera
Rank 2: Etapa 2, después de la barrera
Rank 1: Etapa 2, después de la barrera
```

1. Antes de la barrera cada proceso imprime su etapa 1 sin un orden fijo y luego se queda trabajando un tiempo distinto según su rank. Después de la barrera todos imprimen la etapa 2 casi al mismo tiempo, porque ninguno avanza hasta que el rank 3, el más lento, llega a la barrera.
2. MPI_Barrier sirve para sincronizar a todos los procesos de un comunicador en un mismo punto del programa, asegurando que una etapa termine en todos antes de empezar la siguiente.

Parte 3

```
fabim@JostMachine:/mnt/c/Users/fabim/OneDrive/Desktop/UVG/Semestre8/CompuParalela/Hoja2$ mpicc parte3.c -o parte3
fabim@JostMachine:/mnt/c/Users/fabim/OneDrive/Desktop/UVG/Semestre8/CompuParalela/Hoja2$ mpiexec -n 4 ./parte3
Proceso 3 recibió el valor: 42
Proceso 0 recibió el valor: 42
Proceso 1 recibió el valor: 42
Proceso 2 recibió el valor: 42
```

1. MPI_Bcast envía el valor que tiene el proceso raíz, en este caso el coordinador con rank 0, a la misma variable de todos los demás procesos. MPI_INT es el tipo de dato de MPI que indica que lo que se envía es un entero, para que sepa cuántos bytes mover y cómo interpretarlos.
2. Con 99999 no sucede nada distinto, todos los procesos reciben el mismo valor porque sigue estando dentro del rango de un int, que llega hasta 2 147 483 647.

```
fabim@JostMachine:/mnt/c/Users/fabim/OneDrive/Desktop/UVG/Semestre8/CompuParalela/Hoja2$ mpicc parte3.c -o parte3
fabim@JostMachine:/mnt/c/Users/fabim/OneDrive/Desktop/UVG/Semestre8/CompuParalela/Hoja2$ mpiexec -n 4 ./parte3
Proceso 3 recibió el valor: 99999
Proceso 0 recibió el valor: 99999
Proceso 1 recibió el valor: 99999
Proceso 2 recibió el valor: 99999
```

3. El coordinador es el único que asigna el valor y MPI_Bcast se encarga de repartir una copia a cada proceso, de modo que al final todos tienen el mismo valor sin necesidad de enviarlo uno por uno.

